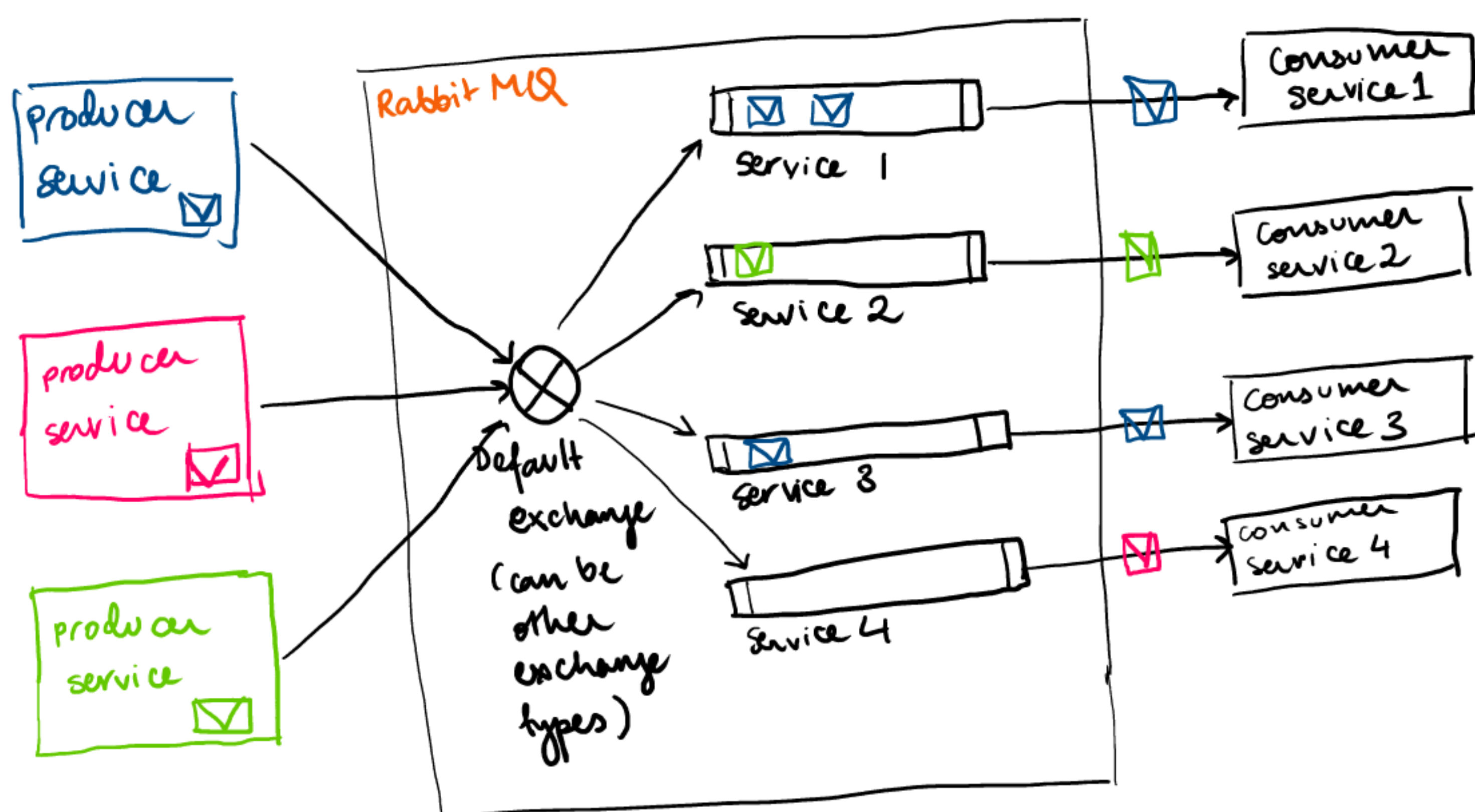


# RabbitMQ

17 December 2025

19:54

→ Literally a queue that has AMQP.



## Message loss:

Producer sends a message but connection drops

\* lost if producer hasn't enabled publisher confirms:

python

```
channel.confirm_delivery()
```

if this is on, producer waits for CONFIRM or NACK from broker & can resend if NACK / nothing happens.

Queue doesn't persist messages; broker restart ⇒ Message loss.

\* Can make sure messages are persistent (stored on disk rather than in memory):

python

```
queue_declare(durable=True)
```

→ This means the queue itself (structure, name, metadata) is saved to disk.

The queue will still exist after a RabbitMQ restart.

It doesn't guarantee message survival. For that you need this too:

python

```
basic_publish(delivery_mode=2)
```

Consumer receives message but crashes before ack.

If `auto_ack=True`, message lost forever  
If `manual_ack=True`, message is not acknowledged → RabbitMQ requeues it. These are set by the consumer.

If the consumer does `basic_nack(requeue=False)`

because `requeue=False`, message is lost unless Dead Letter Queue (DLQ) is set up.

Dead Letter Queue has  
• expired messages (TTL)  
• rejected messages  
• queue overflow  
Help avoid data 'vanishing'

This applies to only one message

## If the broker runs out of memory or disk rabbitMQ can also use:

1) Flow control:

When rabbitMQ notices limits for

- memory usage
- disk space
- file descriptors
- queue backlogs

are exceeded, it can slow down producers.

It stops accepting new messages from producers.

Producers get blocked or connections are paused.

As queues clear and resources stabilize, RabbitMQ resumes accepting messages.

2) You can also limit how many unacked messages are delivered to consumers (BACK PRESSURE):

- RabbitMQ delivers only 1 unacked message at a time per consumer
- consumers can't be flooded with tasks. Prevents consumer overload → prevents system collapse

python

```
channel.basic_qos(prefetch=N)
```

if  $N=1$ , rabbitMQ will deliver only 1 unacknowledged message at a time per consumer.

set by the consumer.

QoS = 'quality of service'

If tasks are short & you want high throughput, use `prefetch > 1`, else `= 1`.

Or if task is I/O bound, then `prefetch > 1` because the producer might be idle for a while.